

OS and Kernel

1. הדיאלקטיקה הסמנטית: האם מדובר במושגים נפרדים?

אלו מושגים שונים מהותית, אך קיימת ביניהם זיקה היררכית ופונקציונלית הדוקה.

הבלבול בספרות נובע משלוש סיבות עיקריות:

1. לעיתים קרובות משתמשים בחלק (ה-Kernel) כדי לתאר את השלם (ה-OS), שכן הליבה היא המרכיב העיקרי של מערכת ההפעלה.
ללא ליבה, אין מערכת הפעלה; אך ללא כלי עזר (Utilities), המערכת אינה שימושית למשתמש הקצה.
2. **התפתחות המיקרו-ליבה (Microkernel) מול המונוליט (Monolith):** בארכיטקטורות מונוליטיות (כמו Linux), הגבול בין שירותי המערכת לליבה מטושטש פיזית, מה שמעודד שימוש חופשי במונחים. במיקרו-ליבות (כמו L4 או QNX), ההפרדה חדה וברורה, והשימוש המעורב במושגים נחשב לשגיאה גסה.
3. **ההיבט המסחרי:** חברות כמו מיקרוסופט או אפל משווקות "Operating Systems" (Windows 11, macOS) הכוללות דפדפנים, נגני מדיה וממשקי גרפיים. עבורן, ה-OS היא ה-Product. עבור מהנדס המערכת, ה-OS היא ה-Environment.

מסקנה: השימוש ה"מתחלף" בספרות הוא אכן לגיטימי בהקשרים פופולריים או יישומיים, אך בשיח הנדסי-אקדמי, הוא מהווה חוסר דיוק שיש לשרש אצל סטודנטים בשלב מוקדם.

2. הגדרות אופרטיביות ומדויקות

א. מערכת ההפעלה (Operating System)

מערכת ההפעלה היא **אקו-סיסטם תוכנתי מקיף** המהווה שכבת תיווך (Abstraction layer) בין החומרה למשתמש/ליישום. היא מוגדרת כאוסף של רכיבים הכולל:

- **הליבה (Kernel).**
- **ספריות מערכת (System Libraries):** כגון libc או ה-Win32 API.
- **כלי שירות ותוכניות מערכת (System Utilities):** Shells, מנהלי קבצים, קומפילרים המגיעים כחלק מההפצה.
- **ממשקי משתמש (User Interfaces):** GUI או CLI.

ב. הליבה (Kernel)

הליבה היא **המרכיב המרכזי והחיוני ביותר** במערכת ההפעלה. זוהי התוכנית הראשונה הנטענת לזיכרון (לאחר ה-Bootloader) והיא פועלת ברמת ההרשאות הגבוהה ביותר של החומרה (Kernel Mode / Ring 0).

תפקידה המזוקק הוא **ניהול משאבים (Resource Management) וגישור (Mediation)**. היא האחראית הבלעדית על ניהול ה-CPU, הזיכרון הפיזי, והגישה להתקני הקלט/פלט.

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 1

3. ההבחנה האונטולוגית והפונקציונלית

ההבדל בין ה- OS ל- Kernel ניתן לזיקוק לנקודות הבאות:

קריטריון	הליבה (Kernel)	מערכת ההפעלה (OS)
מרחב ריצה	Kernel Space בלבד.	Kernel Space + User Space.
מטרה עיקרית	ניהול חומרה והפשטתה (Abstraction).	אספקת סביבת עבודה מלאה למשתמש וליישום.
אינטראקציה	מתקשרת עם החומרה ועם System Calls.	מתקשרת עם המשתמש (GUI/CLI) ועם היישומים.
גודל ומורכבות	יחידה קומפקטית יחסית (בלוגיקה, לא בהכרח בשורות קוד).	מערך עצום של תוכנות, ספריות ומנהלי התקנים.
קיום	ישות תוכנתית אחת (Binary) רציפה בזיכרון.	אוסף של קבצים ותהליכים נפרדים.

הדגשה טכנית: הליבה אינה מבצעת עבודה עבור עצמה, אלא מגיבה לבקשות (System Calls) או לאירועי חומרה (Interrupts).

4. ה-Kernel כ"מכונה מורחבת" מול ה-OS כ"סביבת עבודה"

בעולם האקדמי, אנו מגדירים את הליבה (Kernel) כישות הסטטית היחידה שרצה בתוך Privileged Mode. ה-Operating System, לעומתה, היא ישות דינמית המשתרעת על פני שני מרחבי כתובות (Address Spaces).

א. הליבה כמנהל משאבים (Resource Management)
הליבה פועלת במונחים של הפשטה (Abstraction). אם המעבד מבין רק פקודות מכונה, הליבה "ממציאה" עבור המשתמש את המושג "תהליך" (Process). אם הדיסק הקשיח מבין רק סקטורים ומגנטים, הליבה "ממציאה" את המושג "קובץ".

- המנגנון (Mechanism): הליבה מספקת את ה-Context Switching, את ניהול ה-Page Tables ואת ה-Interrupt Handlers.
- המדיניות (Policy): ה-OS (דרך רכיבי User-space או הגדרות ליבה) קובעת מי יקבל את המשאב ומתי.

ב. מערכת ההפעלה כחבילת תוכנה (Software Suite)
מערכת ההפעלה המודרנית היא למעשה Framework. היא כוללת ספריות (Libraries) שבלעדיהן ה-Kernel אינו נגיש למפתח הממוצע. למשל, ה-Kernel של לינוקס מספק את ה-write System Call(), אך ה-OS מספקת את ה-Standard C Library (libc) שמאפשרת למפתח להשתמש ב-printf(). ההבדל הזה הוא קריטי להבנת שרשרת הקריאות במערכת.

5. ההבדלים הטכניים

כדי להמחיש את ההבדל, יש לבחון את המערכת דרך שלוש עדשות:

א. היררכיית ההרשאות (Protection Rings)

בארכיטקטורת x86-64, הליבה שוכנת ב-Ring 0. היא היחידה שמסוגלת לבצע הוראות מיוחדות (כמו HLT או שינוי רגיסטרי בקרה של ה-MMU). שאר מרכיבי מערכת ההפעלה רצים לרוב ב-Ring 3.

אם רכיב ב-OS קורס, המערכת עשויה לשרוד.

אם ה-Kernel קורס, מתרחש Fatal System Error (ב-Windows או Kernel Panic בלינוקס).

ב. ניהול מרחב הכתובות (Address Space Isolation)

הליבה ממופה לתוך מרחב הכתובות של כל תהליך במערכת (בדרך כלל בחלקו העליון של הזיכרון הווירטואלי), אך הגישה אליה חסומה ברמת החומרה עבור קוד המשתמש. מערכת ההפעלה, כיישות רחבה, מורכבת ממאות מרחבי כתובות נפרדים (תהליכי שירות) המתקשרים ביניהם באמצעות IPC (Inter-Process Communication).

ג. אופן הפעולה (Execution Flow)

- **הליבה היא Reactive:** היא "ישנה" עד שקורה אירוע (Hardware Interrupt או Exception/Trap). היא אינה "רצה" במובן הרגיל של לולאת תוכנית, אלא היא אוסף של שגרות שירות.
- **מערכת ההפעלה היא Proactive:** היא כוללת תהליכי מערכת (כמו ה-Init בלינוקס או smss.exe ב-Windows) שהם תהליכים לכל דבר, בעלי לולאות ריצה משלהם, המנהלים את מצב המערכת הכללי.

דוגמה:

כאשר קוראים לפונקציה open ב-C:

- אתה קורא לפונקציה ששייכת ל-OS (ספריית glibc).
- ה-glibc מבצעת הוראת SYSCALL.
- באותו רגע, ה-Kernel משתלט על המעבד, עובר ל-Ring 0, בודק הרשאות קבצים, וניגש פיזית לדיסק.
- התוצאה חוזרת ל-glibc, והיא מחזירה אותה לתוכנית שלך.

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 4

Extended Machine

מערכת ההפעלה (OS) כ'מכונה מורחבת' (Extended Machine)

מערכת ההפעלה היא האמצעי ליצירת אשליה עבור המתכנת והמשתמש : אשליה של מכונה ידידותית, יציבה ובלעדית. במציאות, החומרה הפיזית היא מורכבת, מוגבלת, ואינה סלחנית. מערכת ההפעלה מפשטת את המציאות הזו ומעמידה לרשות התוכנה **משאבים וירטואליים רציפים ובלתי-מוגבלים לוגית**. ללא אשליה זו – אותה אנו מכנים "המכונה המורחבת" – החומרה המשוכללת ביותר נותרת אוסף של רכיבים חסרי תועלת שאינם ניתנים לניהול ברמת הקוד.

הליבה (Kernel) כסמכות הבלעדית

בתוך ה-Ecosystem הזה, הליבה (Linux Kernel 6.14) היא הגוף היחיד בעל הסמכות המוחלטת. מבחינה אופרציונלית, הקרנל הוא אוסף הקוד המורץ על המעבד במצב הפריבילגי הגבוה ביותר (Ring 0 / Supervisor Mode). הוא הישות היחידה שיכולה לבצע הוראות מיוחסות (Privileged Instructions). בעוד שמערכת ההפעלה כולה מגדירה את החוויה והסביבה (המכונה המורחבת), הליבה היא המנוע שמבצע את ה"הונאה" הזו אל מול החומרה.

היחסים הסימביוטיים באקו-סיסטם

הרכיבים השונים – הקרנל, ספריות הבסיס (glibc), ה-Bash, ותוכניות השירות – אינם ישויות עצמאיות. הם מקיימים קשר סימביוטי הדוק :

- הקרנל מספק את התשתית הפיזית והבידוד.
- ה-glibc מתרגמת את בקשות המשתמש לממשק המובן לקרנל (ABI).
- ה-Bash (כתוכנית מתווכת) מאפשרת למשתמש האנושי להפעיל את המכונה הזו.

קריסה או חוסר התאמה באחד מהמרכיבים הללו משביתה את ה-Ecosystem כולו. הבנת המערכת כולה מחייבת לראות כיצד כל רכיב תלוי בחוזה התוכנה (ABI/API) של הרכיב שמתחתיו.

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 5

ה- Extended Machine כציר מרכזי

הספרות האקדמית נוטה להגדיר קרנל דרך "תפקידים" (Resource Manager, Abstraction Layer, Scheduler).

הבעיה היא שזו 'רשימת מכולת'.

אנחנו נשתמש ב- Extended Machine כציר מרכזי, דבר ש'יגרום' להפוך את הדיון מ"מה הליבה עושה" ל"איזו אשליה הליבה בונה":

- **מנהל התהליכים (Scheduler):** הוא לא "מתזמן מעבדים", הוא **הבונה של אשליית הבלעדיות**. הוא גורם לכל תוכנית "לחשוב" שהיא רצה על מעבד משלה, למרות שהיא למעשה חולקת אותו עם מאות תהליכים אחרים.
- **מנהל הזיכרון (Memory Manager):** הוא לא "מנהל דפים ו-Page Tables", הוא **הבונה של אשליית המרחב הרציף והבלעדי**. בלעדיו, המתכנת היה צריך להתמודד עם שברים בזיכרון הפיזי, עם התנגשויות בין תהליכים ועם הגבלות פיזיות.
- **מערכת הקבצים (File System):** היא לא "דרייבר לדיסק", היא **הבונה של אשליית העמידות והארגון**. היא הופכת סקטורים מגנטיים/חשמליים על הדיסק למבנה לוגי של היררכיה ושמות, מה שמאפשר לאשליה של המכונה המורחבת להתקיים גם כשהחשמל כבוי.

היתרון הפדגוגי: מניעת "אמורפיות"

הגדרת הקרנל דרך הפוטנציאליות שלו היא "מגמגמת".

היא משאירה אותנו עם שאלות כמו "אז איפה עובר הגבול?".

ההגדרה שקבענו (קוד ב-0 Ring) היא **בינארית**. היא חומרית.

היא לא משאירה מקום לפרשנות.

ברגע שנבין את ההפרדה המכנית הזו, נבין שהקרנל הוא "השומר שעל החומה" – כל מה שמעבר

לחומה (Ring 0) הוא קוד של ה- Extended Machine, וכל מה שמתחתיה הוא ה- Hardware.

תפקידי הקרנל

להלן תפקידי הקרנל, כשהם קשורים ל-Reference Point שלנו (יצירת ה-Extended Machine):

1. מנהל התהליכים (Process Management)

- **התפקיד הטכני:** ניהול ה-Context Switching, Context, ותזמון (Scheduling).
- **התקשור ל-Extended Machine:** זהו הבונה של **אשליית הבלעדיות**. המכונה האמיתית היא סדרתית במעבד אחד; ה-Extended Machine "מבטיחה" למתכנת שהתוכנית שלו רצה לבדה, רצוף ובלי הפרעות. ה-Scheduler הוא המנגנון שמוודא שהמשתמש לעולם לא ירגיש את ה"קרטועים" של המעבר בין תהליכים.

2. מנהל הזיכרון (Memory Management)

- **התפקיד הטכני:** Paging, Page Tables, הקצאת זיכרון, ו-Virtual Address Space.
- **התקשור ל-Extended Machine:** זהו הבונה של **אשליית המרחב הרציף והבלעדי**. במכונה האמיתית, הזיכרון מקוטע, מפוזר ב-Physical Pages, ומשותף עם תהליכים אחרים. ה-Memory Manager יוצר עבור התוכנית מרחב כתובות וירטואלי, רציף, שבו היא יכולה לכתוב בלי לחשוש שתדרוס תהליך אחר.

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 6

3. מנהל התקני הקלט/פלט (Device Management)

- **התפקיד הטכני:** ניהול דרייברים (Drivers), Interrupt Handling, ו-MMIO/DMA.
- **התקשור ל-Extended Machine:** זהו הבונה של **אשליית השקיפות**. החומרה היא רועשת, לא אחידה, ודורשת פקודות מורכבות (Timing-sensitive). המכונה המורחבת מסתירה את כל זה מאחורי ממשקים פשוטים (כמו File Descriptors). המשתמש "כותב לקובץ" או "קורא מרשת" – הוא לא צריך לדעת איך הבקר הספציפי מדבר עם החומרה.

4. מערכת הקבצים (File System)

- **התפקיד הטכני:** ניהול היררכיית קבצים, הרשאות (Permissions), וניהול עמידות נתונים (Persistence).
- **התקשור ל-Extended Machine:** זהו הבונה של **אשליית העמידות והארגון**. הדיסק הפיזי הוא סדרה ארוכה של בלוקים. המערכת המורחבת מציגה "עץ היררכי" נוח לוגית. היא מספקת למשתמש את האשליה שהנתונים שלו "נשמרים" בצורה מסודרת, גם כאשר ברמה הפיזית הם מפוזרים על פני פלטורות מגנטיות או תאי פלאש.

5. מערך ההגנה והבידוד (Security & Protection)

- **התפקיד הטכני:** הגדרת ה-Privilege Levels (Ring 0/3), ניהול הרשאות, ו-Sandboxing.
- **התקשור ל-Extended Machine:** זהו הבונה של **אשליית היציבות והאמינות**. כדי שהמכונה המורחבת תהיה "אמינה", היא חייבת להיות מוגנת מפני המשתמש. אם תוכנית אחת יכולה לקרוס ולגרום לכל המכונה (המערכת) להפסיק לעבוד, האשליה מתנפצת. הבידוד הוא מה שמאפשר לסטודנטים ולמשתמשים להריץ יישומים מורכבים בלי לפחד שהם "יהרסו את המחשב".

סיכום המבט הארכיטקטוני:

לפי הגישה הזו, הקרנל אינו "מנהל משאבים" במובן הסתמי. הוא **מפעל של אשליות**. כל שורת קוד בקרנל שמבצעת משימה, עושה זאת כדי לשמר את התוקף של המכונה האבסטרקטית שהבטחנו למשתמש.

הקשר הסימביוטי בתוך ה-Ecosystem של Linux 6.14

הקשר הסימביוטי בתוך ה-Ecosystem של Linux 6.14 אינו סתם "תלות" – זוהי **מערכת אקולוגית הנדסית** שבה כל רכיב מתוכנן להזין, להגן ולייעל את ה"אשליה" שמייצר הרכיב שלידו. אם נפרק את הקשרים הללו, נראה כיצד ה-Extended Machine מתפרקת למרכיביה הבסיסיים. להלן הניתוח המקיף של היחסים הסימביוטיים:

1. החוזה שבין ה-Kernel ל-glibc (הגשר המכני)

- זהו הקשר הסימביוטי הקריטי ביותר. ה-Kernel מגדיר את ה-System Call Interface, אך הוא נגיש רק דרך פקודת SYSCALL בשפת Assembly.
- **התרומה של glibc:** היא "עוטפת" את ה-Syscalls המורכבים והבלתי-נוחים (שדורשים העברת אוגרים מדויקת) בתוך פונקציות C ידידותיות.
- **הקשר למכונה האבסטרקטית:** ה-Extended Machine מבטיחה למתכנת "פונקציה לפתיחת קובץ".

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 7

ה- glibc מתרגמת את האשליה הזו לביצוע מכני של הוראות מעבד, והקרנל מוודא שהאשליה נשמרת ברמת הרשאות המשתמש.
 ללא ה- glibc, המכונה האבסטרקטית הייתה "קשה מדי" לשימוש ; ללא הקרנל, ה- glibc הייתה רק אוסף של פקודות ריקות !!.

2. הקשר שבין ה-Shell ל-Kernel (הערוץ הניהולי)

- ה- Bash הוא לא רק "ממשק", הוא הסוכן המבצע של האשליה.
- הדינמיקה:** המשתמש מזין string (פקודה). ה- Bash מבצע fork() ו- execve(). פעולות אלו הן קריאות מערכת שמפעילות את מנהל התהליכים בקרנל.
- הקשר למכונה האבסטרקטית:** ה- Bash משתמש בקרנל כדי ליצור את המשאבים הווירטואליים של התהליך החדש (מרחב כתובות, תיאורי קבצים). כאן הסימביוזה ברורה: ה- Bash זקוק לקרנל כדי "לגדול" (להוציא תהליכים לפועל), והקרנל זקוק ל- Bash כדי לקבל הוראות משתמש.

3. מנהלי התקנים (Drivers) והחומרה (הערוך המזין)

- הדינמיקה:** הדרייבר הוא חלק מהקרנל, אך הוא תלוי לחלוטין ב- Interrupt Controller (חומרה) כדי לדעת מתי נתונים הגיעו מהעולם החיצון.
- הקשר למכונה האבסטרקטית:** הדרייבר אחראי ל"תרגום" של החומרה הפיזית לאובייקט לוגי (כמו Socket או File Descriptor) שהמכונה האבסטרקטית מציגה למשתמש.

המבנה הסימביוטי

יחס גומלין	המרכיב ה"מזין"	המרכיב ה"ניזון"	האשליה הנשמרת
חזיה ABI	Kernel	glibc	אחידות תכנותית
תהליכים	Bash	Kernel	בלעדיות הרצה
ניהול חומרה	Drivers	Kernel	שקיפות חומרתית

מצבי המעבד ומרחבי הזיכרון

מצבי מעבד, מרחבי זיכרון והגדרת הקרנל האופרציונלית

כדי שהמכונה המורחבת (Extended Machine) תפעל, החומרה חייבת לאכוף גבולות. ב-x86_64, גבולות אלו ממומשים באמצעות **מצבי המעבד (CPU Modes)**.

- **מצב המעבד (CPL - Current Privilege Level):** המעבד מנהל אוגר פנימי (חלק מה-CS Segment Selector) המגדיר את רמת ההרשאה הנוכחית.
 - **Ring 0:** המצב הפריבילגי הגבוה ביותר, בו רץ הקרנל. במצב זה, ה-CPL הוא 0.
 - **Ring 3:** המצב המוגבל, בו רצים יישומי המשתמש.
- **Kernel Space vs. User Space:** זהו הנגזרת של מצב המעבד. ב-Linux 6.14, מרחב הכתובות הווירטואלי מחולק:
 - **User Space:** אזור זיכרון שאינו נגיש ב-Ring 0 ללא הגדרה מכנית מיוחדת, ומוגבל לגישה ב-Ring 3.
 - **Kernel Space:** אזור זיכרון שמוגן מפני גישה ב-Ring 3. הניסיון לגשת אליו ב-Ring 3 יגרור מיד Page Fault (באמצעות ביט ה-User/Supervisor בטבלאות הדפים).

הגדרה אופרציונלית של הקרנל:

"ה-Kernel הוא אוסף כל הקוד (למעט ה-Bootloader) אשר בעת ביצועו, המעבד נמצא ב-CPL=0".

הגדרה זו היא קריטית כי היא מחברת בין הקוד הסטטי שעל הדיסק לבין המצב הדינמי [= בזמן ריצה].

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 9

Device Drivers & ISRs

כעת, בחלק הזה של התיאור/דיון, אנו עוברים לבסס את המעבר מה"סמכות" (הקרנל) ל"ביצוע" (דרייברים ו-ISRs), תוך שמירה על היררכיית ה-Ring 0.

כדי שה-Extended Machine תקיים את האשליה של משאבים זמינים, הקרנל חייב "לצאת" מגבולות ה-CPU ולקיים אינטראקציה עם החומרה הפיזית. הדרייברים וה-ISRs הם האמצעים שדרכם הליבה מבצעת אינטראקציה זו.

מנהלי התקנים (Drivers) – ה"מתרגמים"

מנהל התקן הוא רכיב קוד (המהווה חלק מהקרנל) האחראי על תרגום הצרכים הלוגיים של המכונה האבסטרקטית לפעולות פיזיות בחומרה.

- **הסמכות המכנית:** הדרייבר רץ ב-Ring 0. הוא אינו מוגבל; הוא יכול לגשת לכל כתובת בזיכרון ולבצע פקודות I/O מיוחדות (כמו פקודות IN/OUT ב-x86 או גישת MMIO).
- **החשיבות:** ללא דרייברים, הקרנל הוא מעין "מוח" ללא גוף. הדרייבר הוא שמתרגם בקשה לוגית (כמו "שלח מנה ברשת") לרצף חשמלי מורכב ברמת ה-NIC (Network Interface Card).

ISRs (Interrupt Service Routines) – "דפיקות הדלת" של החומרה

פסיקה (Interrupt) היא האירוע שבו החומרה "מושכת במושכות" של המעבד כדי לקבל תשומת לב מיידית. ה-ISR הוא הקוד שנקרא בתגובה לאירוע זה.

- **הקריטיות המכנית:** כאשר מגיעה פסיקה, המעבד מפסיק את עבודתו השוטפת וקופץ לכתובת המוגדרת ב-IDT (Interrupt Descriptor Table). ה-ISR חייב לרוץ ב-Ring 0, מכיוון שהוא מנהל את הגישה הראשונית לחומרה.
- **האטומיות והמורכבות (Atomic Execution):** ה-ISR מתאפיין באופי אטומי: בעת כניסתו לביצוע, המערכת מבצעת באופן אוטומטי (או שה-ISR עושה זאת מיידית באמצעות הוראת CLI) חסימה של פסיקות אחרות – Masking. זהו רגע מכריע במורכבות של ה-Ring 0: הליבה נמצאת במצב שבו היא "עיוורת" לאירועים חיצוניים אחרים כדי להבטיח את תקינות הטיפול באירוע הנוכחי. זוהי החלטה הנדסית המאזנת בין הצורך בטיפול מהיר ומונע-שיבושים לבין הסיכון ב-Deadlock או באובדן מידע מהעולם החיצון. אם ה-ISR לא יחזיר שליטה בזמן, המערכת כולה תקפא.
- **יחסי הגומלין:** ה-ISR הוא ה"זרוע" שמאפשרת לליבה להגיב בזמן אמת. תחשוב על זה כעל "דפיקה בדלת": ה-ISR הוא המענה המיידי שפותח את הדלת ומוודא שהחומרה תטופל לפני שהמצב יאבד.

הסימביוזה ב-Ring 0:

הן הדרייברים והן ה-ISR חולקים את אותה "סמכות" (Ring 0) ואת אותו **Kernel Stack**. זהו מצב של סימביוזה מלאה:

1. **אמון מוחלט:** הליבה "סומכת" על הדרייברים שיתנהגו כשורה. באג בדרייבר משמעו באג בליבה, כיוון שהם חולקים את אותו מרחב כתובות ואותה רמת הרשאה.
2. **הגנה על האשליה:** הם הזרועות המבצעות של ה-Extended Machine. כל אינטראקציה של המשתמש עם "קובץ" או "רשת" עוברת בסופו של דבר דרך הדרייברים וה-ISR הללו.

קופסת ידע: ה-Bootloader (מאתחל המערכת)

ה-Bootloader הוא רכיב תוכנה הכרחי לקיומה של המערכת. מטרתו היא לגשר על הפער שבין החומרה ה"עירומה" לבין המצב שבו הקרנל יכול להתחיל לפעול.

שלבי האתחול:

1. **Firmware ו-POST:** עם הדלקת המחשב, ה-Firmware (תוכנה צרובה ברמת החומרה, כגון UEFI או Legacy BIOS) מבצעת **POST (Power-On Self-Test)** – סדרת בדיקות אבחון עצמי של רכיבי החומרה (מעבד, זיכרון, בקרים). רק לאחר סיום מוצלח של ה-POST, עוברת השליטה ל-Bootloader.
2. **טעינה (Loading):** ה-Bootloader מאתר ב-NVM (Non-Volatile Memory) את קובץ ה-Executable בפורמט ה-ELF (Executable and Linkable Format), המכיל את תוכנית הקרנל. הוא טוען את הקובץ לזיכרון ה-RAM.
3. **אתחול המכונה (System Initialization):** ה-Bootloader מבצע שינוי במצב המעבד. הוא בונה את ה-Page Tables הראשוניות, מגדיר את ה-GDT (טבלת הסגמנטים) ומעביר את המעבד ממצב 16-ביט (Real Mode) למצב 64-ביט (Long Mode).
4. **העברת השליטה:** לאחר שהכין את סביבת העבודה (הגדרת RSP - ה-Stack המוקדם, והכנת אוגרי הבקרה), הוא קופץ לכתובת הכניסה (Entry Point) של הקרנל.

ללא ה-Bootloader, הקרנל הוא רק קובץ בינארי המאוחסן על ה-NVM. הוא בונה את התשתית הראשונית שעל גביה הקרנל ימשיך לבנות את ה-Extended Machine.

(הערה: המכניקה של טעינת קבצי ELF והקפיצה לכתובת ה-Entry Point יתוארו ויוסברו בהמשך הקורס).

המורכבות הכמותית: LOC והטרוגניות השפות

כדי להבין מדוע מערכת הפעלה אינה רק אוסף של פקודות, אלא מבנה הנדסי מורכב מאין כמותו, עלינו לבחון את ה"מסה" של הקוד (LOC - Lines of Code) ואת הבחירה בשפות התכנות. להלן סקירה של המורכבות הכמותית, המהווה את הגשר (Hook) למעבר מהתכנון התיאורטי אל המציאות של ה-SDLC.

1. סדרי גודל (LOC): ה-Kernel כמרכז הכובד

- לינוקס 6.14 מורכבת ממיליוני שורות קוד (מעל 30 מיליון שורות במקורות הרשמיים, כולל מנהלי התקנים). כדי להבין את ה-Ecosystem, עלינו להבדיל בין סדרי הגודל:
 - הליבה (Kernel):** ה-TCB (Trusted Computing Base) מונה מיליוני שורות קוד. המורכבות כאן אינה רק כמותית אלא איכותית – כל שורת קוד כאן רצה ב-Ring 0, מה שאומר שבאג אחד יכול לערער את יציבות המערכת כולה.
 - ספריות המערכת (glibc) ותוכניות העזר (Coreutils):** אלו מוסיפות עוד מיליוני שורות קוד המיועדות ל-User-Space. הן מהוות את ה"תיווך" (Abstractions) שמאפשר למערכת להיות שמישה.
 - ההשלכה ההנדסית:** המורכבות הזו הופכת את ה-SDLC מ"רצוי" ל"הכרחי". אי אפשר לתחזק מערכת בסדר גודל כזה ללא תיעוד קפדני, בלי תהליכי Code Review מחמירים, ובלי ניהול גרסאות (Version Control) מבוצר.

2. בחירת שפת המימוש: הגבול שבין האדם לחומרה

הטרוגניות השפות במערכת אינה מקרית – היא משקפת את רמות האבסטרקציה השונות ב-Extended Machine.

• C (Assembly & Kernel):

- שפת C: זוהי שפת ה-Systems Programming האולטימטיבית.
 - היא מעניקה למתכנת מודל שפה הממפה באופן דטרמיניסטי את הקוד לכתובות זיכרון (Memory Layout) ולמבני נתונים (Data Structures).
 - בשונה משפות המסתירות את ניהול המשאבים, שפת C מאפשרת למפתח להגדיר מבני נתונים שגודלם, סדר השדות שלהם (Offset) והקשרם בזיכרון הפיזי מוגדרים מראש וקבועים.
 - בכך, היא מהווה את השכבה שבה הלוגיקה של ה-API מתורגמת ישירות לחישובי כתובות (Addressing) ולביצוע הוראות מול אוגרי המעבד (Registers).
 - היכולת להבטיח את מיקומו של מבנה נתונים בזיכרון היא ההכרח המכני שמאפשר לקרנל לגשת למידע בצורה עקבית, בכל פעם שהמעבד מבצע פקודה.

- Assembly (x86_64): הקרנל חייב להכיל רכיבי Assembly כדי לבצע פעולות הדורשות גישה לאוגרי מעבד או שינוי מצב המערכת שלא ניתן למימוש באמצעות סמנטיקת השפה C.

לדוגמא: שליטה באוגרי בקרה (כמו CR3), ניהול Interrupts, וביצוע ה-SYSCALL עצמו. זוהי השפה היחידה שהחומרה "מבינה" ללא תיווך.

• User-Space (C, C++, Rust, Python): היעילות האנושית

- ברגע שהליבה בנתה את ה-Extended Machine, המפתחים יכולים להשתמש בשפות ברמה גבוהה יותר.
 - כאן, השפה נבחרת לפי הפרודוקטיביות (פיתוח מהיר) ולא לפי השליטה בחומרה.
 - המערכת מספקת להם "סביבה בטוחה" שבה שגיאת זיכרון (כמו Segfault) תגרום לקריסת התהליך בלבד, ולא להשבתת המכונה כולה.

תכנות מערכות, תשפ"ו, סמסטר קיץ

מבוא

עמוד 12

3. התלות בארכיטקטורה (Hardware-Dependent Code)

קריטי להבין שקוד הקרנל אינו "נייטרלי".

הוא כתוב מתוך התחשבות עמוקה ב- ISA (Instruction Set Architecture).

- כל מבנה נתונים בקרנל (כמו ה- task_struct) מעוצב בהתאם לארכיטקטורת המעבד.
- האופן שבו ה- Kernel "רואה" את הזיכרון (Paging) משתנה בין Intel ל- ARM.

קופסת ידע: TCB (Trusted Computing Base)

ה- TCB הוא מושג מפתח באבטחת מערכות המגדיר את אוסף כל מרכיבי המערכת – תוכנה, חומרה וקושחה – שאחראים על אכיפת מדיניות האבטחה.

אם TCB של מערכת נפרץ או מכיל באג, לא ניתן להבטיח יותר את תקינות המערכת.

במערכת לינוקס, הליבה (Kernel) היא ה- TCB המרכזי: היא זו שמבצעת את ההפרדה בין תהליכים, מנהלת את הזיכרון ומגנה על ה- Extended Machine מפני גישה בלתי מורשית.